

Unit Testing Best Practices (JUnit)

1. What are the best practices for writing unit tests using JUnit?

A: Best practices involve making tests **isolated**, meaning each test checks only one thing without affecting others. Use the **AAA structure (Arrange-Act-Assert)** for clear organization. Ensure tests are **fast** and **repeatable** (running them many times gives the same result). Also, only test the public behavior of a class, not its internal private methods.

2. How do you mock static methods in Mockito?

A: Standard Mockito cannot mock static methods by itself. You must use the external feature **Mockito.mockStatic(Class.class)**, which requires the **mockito-inline** dependency. This allows you to define the mock behavior for a static method within a **try-with-resources** block for the duration of that specific test.

3. How do you integrate Mockito with JUnit for full coverage?

A: You integrate Mockito by using the **@ExtendWith(MockitoExtension.class)** annotation (JUnit 5) or **@RunWith(MockitoJUnitRunner.class)** (JUnit 4) on your test class. You then use the **@Mock** annotation to create fake dependencies and **@InjectMocks** to create the object under test, ensuring your service logic is fully isolated.

Large-Scale Test Suites (TestNG)

4. How do you structure large-scale TestNG test suites for CI/CD?

A: You structure large-scale suites using **TestNG Groups** and the **testng.xml** file.

1. **Groups:** Assign clear tags like smoke, regression, or integration to your test methods.
2. **testng.xml:** Configure the XML file to define different **test suites** (e.g., a "Smoke Test Suite") that select and run tests based only on those specific group tags. This allows the CI/CD pipeline to quickly run only the necessary subset of tests.

5. What is the role of filters in RestAssured, and how do you implement them?

A: The role of **filters** in RestAssured is to intercept and modify the request before it is sent or the response before it is validated. They are used for cross-cutting tasks like logging specific security details or recording response times. You implement a filter by creating a class that implements the **io.restassured.filter.Filter** interface and registering it with the request using **.filter(new MyCustomFilter())**.

6. How do you integrate RestAssured with JUnit/TestNG for full test automation?

A: You integrate RestAssured by writing your API test logic directly inside a standard **@Test** method (from JUnit or TestNG). You use **RequestSpecification** (for setup) and **ResponseSpecification** (for validation) to create reusable blocks of code. The test runner (JUnit/TestNG) executes the methods, while RestAssured handles the HTTP requests and response parsing.

JAVA Interview Buddy